

gh-post-range-diff

Haskell sources - 2026-09-06 - 18 files, 2117 lines

./app/Main.hs	45
./rangediff/src/RangeDiff.hs	137
./rangediff/src/RangeDiff/CommitSha.hs	29
./rangediff/test/RangeDiffSpec.hs	64
./rangediff/test/Spec.hs	1
./rangediff/testlib/RangeDiff/Test/Gen.hs	31
./rangediff/testlib/RangeDiff/Test/Git.hs	30
./rangediff/testlib/RangeDiff/Test/Plan.hs	322
./rangediff/testlib/RangeDiff/Test/Repo.hs	435
./src/GhPostRangeDiff/Git.hs	74
./src/GhPostRangeDiff/GitHub.hs	128
./src/GhPostRangeDiff/Render.hs	46
./src/GhPostRangeDiff/Run.hs	96
./test/GhPostRangeDiff/BasesSpec.hs	258
./test/GhPostRangeDiff/FakeGit.hs	156
./test/GhPostRangeDiff/FakeGitHub.hs	105
./test/GhPostRangeDiff/RenderSpec.hs	159
./test/Spec.hs	1

./app/Main.hs [45 lines]

```

1 module Main where
2
3 import Data.List (unsnoc)
4 import GhPostRangeDiff.Git (abbrev)
5 import GhPostRangeDiff.Git qualified as Git
6 import GhPostRangeDiff.GitHub (Ev (..), Ref (..), gh)
7 import GhPostRangeDiff.GitHub qualified as GitHub
8 import GhPostRangeDiff.Run (Reported (..), run)
9 import RangeDiff qualified
10 import RangeDiff.CommitSha (CommitSha, commitSha)
11 import System.Environment (getArgs)
12
13 main :: IO ()
14 main = do
15     -- Either just a PR number (manual use: report the most recent force-push),
16     -- or a PR number plus the push's before/after SHAs from the pull_request
17     -- payload (CI use: report exactly that push, whatever kind it is).
18     args <- getArgs
19     case args of
20     [pr] -> manual (gh pr)
21     [pr, before, after]
22     | Just b <- commitSha before,
23       Just a <- commitSha after ->
24         report (gh pr) b a
25     | otherwise -> manual (gh pr)
26     _ -> error "usage: gh-post-range-diff <pr> [<before-sha> <after-sha>]"
27
28 -- Manual use: no SHAs on the command line, so derive them from the most recent
29 -- force-push in the timeline and report on that one.
30 manual :: GitHub.Handle -> IO ()
31 manual pr = do
32     heads <- filter ((== Head) . evRef) <$> GitHub.timeline pr
33     case unsnoc heads of
34     Nothing -> putStrLn "No force-push events on this PR. Nothing to diff."
35     Just (_, h) -> report pr (evBefore h) (evAfter h)
36
37 -- Report on a push, and say so where the run had nothing to do. Whatever the
38 -- range-diff takes is read out of the repo the CLI was run in.
39 report :: GitHub.Handle -> CommitSha -> CommitSha -> IO ()
40 report pr oldHead newHead = do
41     reported <- run (Git.git ".") (RangeDiff.git ".") pr oldHead newHead
42     case reported of

```

```

43   Posted -> pure ()
44   AlreadyReported ->
45     putStrLn ("Already reported on " ++ abbrev oldHead ++ ".." ++ abbrev newHead ++
". Nothing to do.")

```

./rangediff/src/RangeDiff.hs [137 lines]

```

1  -- | The range-diff between two versions of a branch, behind a handle, so that
2  -- it can be mocked. `git range-diff` prints it as text; everything here reads
3  -- that text into values, so that whatever reports on it doesn't have to work
4  -- with the text.
5  module RangeDiff
6    ( Change (..),
7      Commit (..),
8      CommitMessage (messageText),
9      commitMessage,
10     Interdiff (interdiffText),
11     interdiff,
12     Handle (..),
13     git,
14   )
15  where
16
17  import Data.Bifunctor (first)
18  import Data.Maybe (fromMaybe)
19  import RangeDiff.CommitSha (CommitSha, commitSha, shaText)
20  import System.Process (readProcess)
21
22  -- | The subject line of a commit, as `git range-diff` prints it. Holds no
23  -- newline of its own; see 'commitMessage'.
24  newtype CommitMessage = CommitMessage {messageText :: String}
25    deriving (Eq, Show)
26
27  -- | Read a commit message, keeping only the first line. git stores a whole
28  -- message but range-diff prints the subject alone, on the header line, so that
29  -- is the only part there is to read here.
30  commitMessage :: String -> CommitMessage
31  commitMessage = CommitMessage . takeWhile (/= '\n')
32
33  -- | The interdiff git printed under a @!@ entry, de-indented so its own +/-
34  -- sit in column 0. Always ends with a newline; see 'interdiff'.
35  newtype Interdiff = Interdiff {interdiffText :: String}
36    deriving (Eq, Show)
37
38  -- | Read an interdiff, adding a newline if it doesn't end with one.
39  interdiff :: String -> Interdiff
40  interdiff = Interdiff . unlines . lines
41
42  -- | What happened to one commit between the old and the new range.
43  data Change
44    = Added
45    | Removed
46    | Unchanged
47    | -- | Same commit, different diff. Carries the interdiff git printed
48      -- underneath the header.
49      Updated Interdiff
50    deriving (Eq, Show)
51
52  -- | One commit, as `git range-diff` reports it.
53  data Commit = Commit
54    { cmChange :: Change,
55      -- | The side that still exists: the new sha for =!/>, the old one for <.
56      cmCommitSha :: CommitSha,
57      cmCommitMessage :: CommitMessage
58    }
59    deriving (Eq, Show)
60

```

```

61 -- | A range-diff line is a header iff it is indented by fewer than four
62 -- spaces. git indents the interdiff body by exactly four, and right-aligns the
63 -- commit numbers in a header to the width of the largest one ? so once either
64 -- range reaches ten commits, a single-digit header carries a leading space of
65 -- its own. Only a range of ten thousand commits could pad one out to four.
66 -- A blank line is never a header; it can only have come from the interdiff.
67 isHeader :: String -> Bool
68 isHeader l = case span (== ' ') l of
69   (_, "") -> False
70   (indent, _) -> length indent < 4
71
72 -- | Pair each header with the body lines beneath it. Body lines before the
73 -- first header (which shouldn't happen) are dropped.
74 chunks :: [String] -> [(String, [String])]
75 chunks ls = case dropWhile (not . isHeader) ls of
76   [] -> []
77   (h : rest) -> let (body, more) = break isHeader rest in (h, body) : chunks more
78
79 -- | Drop up to @@ leading spaces, undoing the indent git gives the interdiff.
80 stripIndent :: Int -> String -> String
81 stripIndent n (' ' : s) | n > 0 = stripIndent (n - 1) s
82 stripIndent _ s = s
83
84 -- | Split a line into its first @@ space-separated fields and whatever
85 -- follows them. That remainder loses only the single space that separated it
86 -- from field @@.
87 peel :: Int -> String -> Maybe ([String], String)
88 peel 0 s = Just ([], s)
89 peel n s = case span (/= ' ') (dropWhile (== ' ') s) of
90   ("", _) -> Nothing
91   (f, rest) -> first (f :) <$> peel (n - 1) (drop 1 rest)
92
93 -- | Read one commit from its header line, e.g.
94 --
95 -- > 1: 5ed838c ! 1: f69a2d3 second commit
96 --
97 -- The marker is one of @=@ (unchanged), @!@ (same commit, different diff),
98 -- @>@ (added), @<@ (removed). For an added commit the old side is @- -----@;
99 -- for a removed one the new side is. Everything past the fifth field is the
100 -- commit message, taken verbatim.
101 mkCommit :: String -> [String] -> Commit
102 mkCommit l body = case peel 5 l of
103   Just ([_, oldSha, [m], _, newSha], subj) -> case m of
104     '>' -> commit Added newSha subj
105     -- A removed commit only exists on the old side; everything else has a
106     -- new-side sha we can link to.
107     '<' -> commit Removed oldSha subj
108     '=' -> commit Unchanged newSha subj
109     '!' -> commit (Updated (interdiff (unlines (map (stripIndent 4
110   ) body)))) newSha subj
111   _ -> bad
112   _ -> bad
113   where
114     -- The side we didn't pick is `-----`, so only the one we keep is read.
115     commit change sha subj = Commit
116       change (fromMaybe bad (commitSha sha)) (commitMessage subj)
117       bad = error ("unexpected range-diff header: " ++ l)
118
119 -- | Parse `git range-diff` output, one 'Commit' per header line.
120 parse :: String -> [Commit]
121 parse = map (uncurry mkCommit) . chunks . lines
122
123 -- | Everything there is to ask of a range-diff.
124 newtype Handle = Handle
125   { -- | Line up @oldBase..oldHead@ against @newBase..newHead@: one 'Commit'
126     -- per commit in either range, saying what happened to it.

```

```

125 rangeDiff :: CommitSha -> CommitSha -> CommitSha -> CommitSha -> IO [Commit]
126 }
127
128 -- | Range-diffs taken in the repo in @dir@, as the `git` CLI takes them.
129 git :: FilePath -> Handle
130 git dir = Handle {rangeDiff = gitRangeDiff dir}
131
132 -- Read what `git range-diff` prints for the two ranges.
133 gitRangeDiff :: FilePath -> CommitSha -> CommitSha -> CommitSha -> CommitSha -> IO [
    Commit]
134 gitRangeDiff dir oldBase oldHead newBase newHead = parse <$> readProcess "git" args ""
135 where
136     args = ["-C", dir, "range-diff", range oldBase oldHead, range newBase newHead]
137     range a b = shaText a ++ ".." ++ shaText b

```

./rangediff/src/RangeDiff/CommitSha.hs [29 lines]

```

1 -- | The commit shas a range-diff is taken over, and the ones it reports back.
2 module RangeDiff.CommitSha
3   ( CommitSha (shaText),
4     commitSha,
5     knownSha,
6   )
7 where
8
9 import Data.Char (isDigit)
10 import Data.Maybe (fromMaybe)
11
12 -- | A commit sha, whether abbreviated or written out in full.
13 newtype CommitSha = CommitSha {shaText :: String}
14 deriving (Eq, Ord, Show)
15
16 -- | Read a sha, rejecting anything that isn't one. Git abbreviates to at least
17 -- four hex digits and never past the full forty, and prints them in lowercase.
18 commitSha :: String -> Maybe CommitSha
19 commitSha s
20   | n >= 4, n <= 40, all isHex s = Just (CommitSha s)
21   | otherwise = Nothing
22 where
23     n = length s
24     isHex c = isDigit c || c `elem` ['a' .. 'f']
25
26 -- | Read a sha we already know to be one, because git or GitHub just printed it
27 -- as one.
28 knownSha :: String -> CommitSha
29 knownSha s = fromMaybe (error ("not a sha: " ++ s)) (commitSha s)

```

./rangediff/test/RangeDiffSpec.hs [64 lines]

```

1 -- | Reading the range-diff of one rewrite of a branch: build a repo, rewrite
2 -- the branch in it once, and check that what `git range-diff` is read as says
3 -- what that rewrite did.
4 module RangeDiffSpec (spec) where
5
6 import Data.List (isPrefixOf)
7 import RangeDiff (Change (..), Commit (..), interdiffText)
8 import RangeDiff qualified
9 import RangeDiff.Test.Git qualified as Test.Git
10 import RangeDiff.Test.Repo
11 import Test.Hspec
12 import Test.Hspec.QuickCheck (modifyMaxSuccess)
13 import Test.QuickCheck (Property, conjoin, counterexample, tabulate, (==))
14
15 -- | The marker git prints a commit under. Only good for saying what a rewrite
16 -- covered.
17 marker :: Change -> String
18 marker Unchanged = "="
19 marker (Updated _) = "!"

```

```

20 marker Removed = "<"
21 marker Added = ">"
22
23 -- | What a rewrite rewrote, where it rewrote a commit at all: the hunks its
24 -- interdiff came out with say whether the message was reworted, the file
25 -- amended, or both. Also only good for saying what a rewrite covered.
26 rewrites :: Commit -> [String]
27 rewrites (Commit (Updated patch) _ _) =
28   ["reworted" | metadata `elem` hunks] ++ ["amended" | any (/= metadata) hunks]
29   where
30     hunks = filter ("@" " " `isPrefixOf`) (lines (interdiffText patch))
31     metadata = "@@ Metadata"
32 rewrites _ = []
33
34 spec :: Spec
35 spec = describe "rangeDiff" $
36   -- Every case shells out to build a real repo, so keep the number of them small.
37   modifyMaxSuccess (const 20) $
38     it
39     "reads a change, a sha from the surviving side and a de-indented interdiff per commit"
40     $
41     -- The bases a range-diff is taken over come from the scenario, which
42     -- knows them, so there is no plan this can't be asked of.
43     withRepo (1, 1) (const True) $
44     \repo -> conjoin <$> evolve repo (readBack repo)
45
46 -- | What one rewrite of the branch reads back as, which should be what the
47 -- scenario says it did to every commit, and nothing else.
48 readBack :: Repo -> Rewrite -> IO Property
49 readBack repo rw = do
50   commits <- RangeDiff.rangeDiff (repoRangeDiff repo) oldBase oldHead newBase newHead
51   -- The unparsed range-diff is the first thing you want to look at when the
52   -- model of git's output turns out to be the thing that is wrong.
53   diff <- Test.Git.rangeDiff (repoDir repo) oldBase oldHead newBase newHead
54   base <- baseMovement repo oldBase newBase
55   pure
56     . tabulate "markers" (map (marker . cmChange) want)
57     . tabulate "rewrites" (concatMap rewrites want)
58     . tabulate "base" [show base]
59     . tabulate "commits on the larger side" [bucket (length want)]
60     . counterexample ("range-diff:\n" ++ diff)
61     $ commits === want
62   where
63     Range oldBase oldHead = rwWas rw
64     Range newBase newHead = rwNow rw
65     want = rwRangeDiff rw
66     bucket n = if n >= 10 then "ten or more" else "fewer than ten"

```

```
./rangediff/test/Spec.hs [1 lines]
```

```
1 {-# OPTIONS_GHC -F -pgmF hspect-discover #-}
```

```
./rangediff/testlib/RangeDiff/Test/Gen.hs [31 lines]
```

```

1 {-# OPTIONS_GHC -Wno-orphans #-}
2
3 module RangeDiff.Test.Gen () where
4
5 import Data.Char (isPrint)
6 import Data.List.Extra (trimEnd)
7 import RangeDiff (CommitMessage, commitMessage, messageText)
8 import Test.QuickCheck (Arbitrary
9   (arbitrary, shrink), frequency, listOf1, resize, suchThat)
10 import Test.QuickCheck.Modifiers (getPrintableString)
11
12 -- | A commit message: printable words with the occasional run of backticks,
13 -- which sits mid-line and so must not be read as a code fence.

```

```

14 -- Trimmed of trailing whitespace and never empty, because
15 -- "RangeDiff.Test.Repo" really does commit these. @git commit -m@ cleans the subject u
    p with
16 -- @--cleanup=whitespace@, which strips trailing whitespace, and refuses an
17 -- empty message.
18 instance Arbitrary CommitMessage where
19   arbitrary = commitMessage <$> text `suchThat` (not . null)
20   where
21     text = trimEnd . unwords <$> resize 3 (listOf1 word)
22     word = frequency [(4, getPrintableString <$> arbitrary), (1, pure "```")]
23
24 -- Shrink to another message 'arbitrary' could have made. The 'String'
25 -- shrinker reaches outside that set: it can pick a character that isn't
26 -- printable, drop the last non-space one and leave trailing whitespace
27 -- behind, or empty the message altogether. Trimming a candidate can land it
28 -- back on the original, which would let shrinking loop, so those go too.
29 shrink m = map commitMessage (filter usable (map trimEnd (shrink (messageText m))))
30   where
31     usable m' = not (null m') && all isPrint m' && m' /= messageText m

```

./rangediff/testlib/RangeDiff/Test/Git.hs [30 lines]

```

1 module RangeDiff.Test.Git (git, isAncestorOf, rangeDiff) where
2
3 import Control.Monad (void)
4 import RangeDiff.CommitSha (CommitSha, shaText)
5 import System.Exit (ExitCode (..))
6 import System.Process (readProcess, readProcessWithExitCode)
7
8 -- | Run a git command in the repo in @dir@. Nothing here reads what one prints,
9 -- but it is captured all the same, to keep it out of the test output.
10 git :: FilePath -> [String] -> IO ()
11 git dir args = void (readProcess "git" (at dir args) "")
12
13 -- | Does @a@ come before @b@ on the same line of history?
14 isAncestorOf :: FilePath -> CommitSha -> CommitSha -> IO Bool
15 isAncestorOf dir a b = do
16   (code, _, _) <- readProcessWithExitCode "git" (at dir ["merge-base", "--is-ancestor"
    , shaText a, shaText b]) ""
17   pure (code == ExitSuccess)
18
19 -- | What `git range-diff` prints for @oldBase..oldHead@ against
20 -- @newBase..newHead@, for use in a counterexample.
21 rangeDiff :: FilePath -> CommitSha -> CommitSha -> CommitSha -> IO String
22 rangeDiff dir oldBase oldHead newBase newHead =
23   readProcess "git" (at dir ["range-diff"
    , range oldBase oldHead, range newBase newHead]) ""
24   where
25     range a b = shaText a ++ ".." ++ shaText b
26
27 -- Aim a git command at the repo in @dir@, so that which directory the process
28 -- happens to be in never comes into it.
29 at :: FilePath -> [String] -> [String]
30 at dir args = "-C" : dir : args

```

./rangediff/testlib/RangeDiff/Test/Plan.hs [322 lines]

```

1 -- | The whole of what is going to happen to a repo, worked out as one pure
2 -- value before the repo even exists.
3 --
4 -- A plan says what the branch starts out as (the commits under the fork point
5 -- and the commits above it) and what every rewrite of it does to the version
6 -- before it. Nothing in here has a repo to look at: a commit is no more than
7 -- the message it goes under and the one line of its file it chooses, and a
8 -- rewrite no more than what it does to the commits that are there.
9 -- "RangeDiff.Test.Repo" is what carries a plan out.
10 module RangeDiff.Test.Plan
11   ( Plan (..),

```

```

12 Step (..),
13 Action (..),
14 Committed (..),
15 Shape (..),
16 outcome,
17 everyRewrite,
18 plan,
19 shrinkPlan,
20 )
21 where
22
23 import Control.Monad (replicateM)
24 import Data.Char (isPrint)
25 import Data.List (intercalate)
26 import RangeDiff qualified
27 import RangeDiff.Test.Gen ()
28 import Test.QuickCheck (Gen
    , arbitrary, choose, frequency, getPrintableString, shrink, shrinkList, suchThat)
29
30 -- * What a scenario is going to do
31
32 -- | What a version commits for one commit of the branch: the message it goes
33 -- under, and the one line of its file the version gets to choose.
34 data Committed = Committed
35   { cdMessage :: RangeDiff.CommitMessage,
36     cdLine    :: String
37   }
38   deriving (Eq)
39
40 -- | What one version does to one commit of the version before it, or puts into
41 -- it.
42 data Action
43   = -- | leave the commit it falls to exactly as it is
44     Keep
45   | -- | rewrite the commit it falls to into this
46     Rework Committed
47   | -- | drop the commit it falls to from the branch
48     Drop
49   | -- | put a commit in here that wasn't there
50     Insert Committed
51
52 -- | What one version does to the version before it: something to the stretch
53 -- under the fork point, and something to the stretch above it.
54 --
55 -- Each is a list of actions read against the commits that are there, bottom
56 -- commit first. An 'Insert' falls between two of them and doesn't align
57 -- with an existing commit.
58 data Step = Step
59   { stpBase, stpBranch :: [Action]
60   }
61
62 -- | A whole scenario, worked out before a repo to carry it out in even exists:
63 -- the version of the branch a repo starts out with (the commits under the fork
64 -- point and the commits above it, bottom commit first) and every rewrite of it
65 -- that follows.
66 data Plan = Plan
67   { plBase, plBranch :: [Committed],
68     plSteps :: [Step]
69   }
70
71 -- | The branch as a plan has it partway through: the commits under the fork
72 -- point and the commits above it.
73 data Shape = Shape
74   { shBase, shBranch :: [Committed]
75   }
76   deriving (Eq)

```

```

77
78 -- * Planning one
79
80 -- | A scenario: a first version of the branch, and between @lo@ and @hi@
81 -- rewrites of it.
82 plan :: (Int, Int) -> Gen Plan
83 plan bounds = do
84   base <- baseCommits
85   branch <- branchCommits
86   n <- choose bounds
87   Plan base branch <$> steps n (Shape base branch)
88
89 -- | @n@ rewrites in a row, each worked out from the branch the ones before it
90 -- leave behind.
91 steps :: Int -> Shape -> Gen [Step]
92 steps n sh
93   | n <= 0 = pure []
94   | otherwise = do
95     s <- mutations sh
96     (s :) <$> steps (n - 1) (outcome sh s)
97
98 -- | What the next version does to the branch: something to the stretch under
99 -- the fork point, and something to the stretch above it, and between them
100 -- something a scenario can hold.
101 mutations :: Shape -> Gen Step
102 mutations sh = (Step
103   <$> stretchPlan (shBase sh) <*> stretchPlan (shBranch sh)) `suchThat` pushable sh
104
105 -- | What the next version does to a stretch of branch: something to every
106 -- commit that is there, and now and then a commit or two that weren't,
107 -- wherever they fit.
108 -- Half the time it leaves every commit that is there exactly as it is and only
109 -- puts commits on top of them. That's especially interesting when for the base
110 -- branch, since fast-forwards on the base branch aren't recorded on the GitHub
111 -- timeline, and need to be handled differently by the tool. As such, we want it
112 -- to happen relatively often.
113 stretchPlan :: [Committed] -> Gen [ActionPlan]
114 stretchPlan cs = frequency [(1, onTop), (1, throughout)]
115   where
116     onTop = (map (const Keep) cs ++>) <$> insertions
117     throughout = do
118       bottom <- inserted
119       rest <- mapM step cs
120       pure (bottom ++ concat rest)
121     step c = (:) <$> fate c <*> inserted
122     fate c =
123       frequency
124         [ (4, pure Keep),
125           (3, Rework <$> reworked c),
126           (2, pure Drop)
127         ]
128     -- What goes into one gap: usually nothing, and where something does, now
129     -- and then more than one commit, so that a version can come out with a run
130     -- of commits nobody has seen before standing next to each other.
131     --
132     -- A gap takes about as many commits as it ever did: it fills a little less
133     -- often than it used to, and where it does it now and then brings more than
134     -- one. Every commit put in is another commit to build, so runs any commoner
135     -- than this would cost a scenario real time.
136     inserted = frequency [(9, pure []), (1, insertions)]
137
138 -- | A run of commits put into one gap: one, and now and then more.
139 insertions :: Gen [Action]
140 insertions = (:) . Insert <$> committed <*> frequency [(2, pure []), (1, insertions)]
141

```



```

142 -- | A commit rewritten: reworded, amended, or both. It really is rewritten,
143 -- since a rewrite that left the commit as it was is not one.
144 reworked :: Committed -> Gen Committed
145 reworked c = rewritten `suchThat` (/= c)
146   where
147     rewritten =
148       frequency
149         [ (1, (\m -> c {cdMessage = m}) <$> arbitrary),
150           (1, (\l -> c {cdLine = l}) <$> line),
151           (1, committed)
152         ]
153
154 -- | How many commits the branch starts out with above the fork point. Mostly a
155 -- handful, because every one of them really is committed, but often enough into
156 -- double digits to exercise the leading space git pads a single-digit commit
157 -- number out with once a range holds ten commits.
158 branchCommits :: Gen [Committed]
159 branchCommits = flip replicateM committed =<< frequency [(3, choose (1, 6)), (1
    , choose (10, 12))]
160
161 -- | How many commits it starts out with under the fork point. Often none at
162 -- all, so that it forks straight off the root commit.
163 baseCommits :: Gen [Committed]
164 baseCommits = flip replicateM committed =<< frequency [(1, pure 0), (1, choose (1, 3
    ))]
165
166 -- | A commit a version could carry.
167 committed :: Gen Committed
168 committed = Committed <$> arbitrary <*> line
169
170 -- | The one line of a commit's file that the version carrying it chooses.
171 line :: Gen String
172 line = getPrintableString <$> arbitrary
173
174 -- * Reading a plan without a repo
175
176 -- | The branch a rewrite leaves behind.
177 outcome :: Shape -> Step -> Shape
178 outcome sh s = Shape
    (applied (shBase sh) (stpBase s)) (applied (shBranch sh) (stpBranch s))
179
180 -- | A stretch of branch with a plan's actions read against it, bottom commit
181 -- first.
182 --
183 -- This is "RangeDiff.Test.Repo"'s @apply@ with the repo taken out of it,
184 -- and the two have to agree: what a plan says a stretch becomes is what the
185 -- repo it is carried out in has to end up holding.
186 applied :: [Committed] -> [Action] -> [Committed]
187 applied cs [] = cs
188 applied cs (a : as) = case a of
189   Insert d -> d : applied cs as
190   Keep -> next (\c rest -> c : applied rest as)
191   Rework d -> next (\_ rest -> d : applied rest as)
192   Drop -> next (\_ rest -> applied rest as)
193   where
194     next f = case cs of
195       [] -> error "plan acts on a commit that isn't there"
196       c : rest -> f c rest
197
198 -- | Whether a rewrite is one that could be pushed.
199 --
200 -- It has to leave a range for `git range-diff` to be taken over, so at least
201 -- one commit stays above the fork point. And it has to do something: a version
202 -- that came out as the one before it is a version nobody ever pushed.
203 --
204 -- Whether it did anything is read off the shape alone, which is the careful way

```

```

205 -- round. Two commits carrying the same thing are still different commits in the
206 -- repo, so a rewrite this calls a no-op may well have been a push ? but one it
207 -- lets through always is one.
208 pushable :: Shape -> Step -> Bool
209 pushable sh s = not (null (shBranch sh')) && sh' /= sh
210   where
211     sh' = outcome sh s
212
213 -- | Whether a plan is one 'plan' could have come up with: a branch to take a
214 -- range-diff over throughout, at least one rewrite, and every rewrite one that
215 -- could be pushed.
216 plannable :: Plan -> Bool
217 plannable p =
218   not (null (plBranch p))
219   && not (null (plSteps p))
220   && everyRewrite pushable p
221
222 -- | Whether something holds of every rewrite in a plan, each read against the
223 -- branch the rewrites before it left behind.
224 --
225 -- Every rewrite a plan holds is one that could be pushed, and that is the whole
226 -- of what a plan promises. A property that can only be run on some of them says
227 -- which with this.
228 everyRewrite :: (Shape -> Step -> Bool) -> Plan -> Bool
229 everyRewrite ok p = go (Shape (plBase p) (plBranch p)) (plSteps p)
230   where
231     go _ [] = True
232     go sh (s : ss) = ok sh s && go (outcome sh s) ss
233
234 -- * Shrinking one
235
236 -- | Smaller scenarios to try once one has failed: fewer rewrites, fewer commits
237 -- to start out with, less done by any one rewrite, and smaller messages and
238 -- lines throughout.
239 --
240 -- Nothing here has to keep a plan coherent on its own: a candidate is
241 -- 'normalize'd back into one that says what becomes of the commits that are
242 -- there and nothing else, and thrown away unless it is 'plannable'.
243 shrinkPlan :: Plan -> [Plan]
244 shrinkPlan p =
245   filter plannable . map normalize $
246     [p {plSteps = ss} | ss <- shrinkList shrinkStep (plSteps p)]
247     ++ [p {plBranch = cs} | cs <- shrinkList shrinkCommitted (plBranch p)]
248     ++ [p {plBase = cs} | cs <- shrinkList shrinkCommitted (plBase p)]
249
250 shrinkStep :: Step -> [Step]
251 shrinkStep s =
252   [s {stpBranch = as} | as <- shrinkList shrinkAction (stpBranch s)]
253   ++ [s {stpBase = as} | as <- shrinkList shrinkAction (stpBase s)]
254
255 -- | A milder action to try in one's place. 'Keep' is the mildest there is, and
256 -- everything that falls to a commit shrinks towards it. A commit put in shrinks
257 -- only in what it carries: taking it out of the branch altogether is what
258 -- 'shrinkList' is already doing.
259 shrinkAction :: Action -> [Action]
260 shrinkAction Keep = []
261 shrinkAction Drop = [Keep]
262 shrinkAction (Rework d) = Keep : map Rework (shrinkCommitted d)
263 shrinkAction (Insert d) = map Insert (shrinkCommitted d)
264
265 -- | A smaller thing for a version to commit. The line has to stay printable,
266 -- which the 'String' shrinker does not promise; a 'RangeDiff.CommitMessage'
267 -- keeps itself in range.
268 shrinkCommitted :: Committed -> [Committed]
269 shrinkCommitted c =
270   [c {cdMessage = m} | m <- shrink (cdMessage c)]

```

```

271 ++ [c {cdLine = l} | l <- shrink (cdLine c), all isPrint l]
272
273 -- | The same scenario with every action that has nothing to fall to taken out.
274 --
275 -- Shrinking pulls at one part of a plan at a time, so a candidate can go on
276 -- saying what becomes of a commit that another pull took away: drop a commit
277 -- the branch starts out with, or a commit some rewrite put in, and every
278 -- rewrite that acted on it is left saying one thing too many. This is what
279 -- makes a candidate a scenario again.
280 normalize :: Plan -> Plan
281 normalize p = p {plSteps = go (Shape (plBase p) (plBranch p)) (plSteps p)}
282   where
283     go _ [] = []
284     go sh (s : ss) = s' : go (outcome sh s') ss
285       where
286         s' = Step (trim (shBase sh) (stpBase s)) (trim (shBranch sh) (stpBranch s))
287
288 -- | A stretch's actions with the ones past its last commit dropped. A commit
289 -- put in stays where it was: it is the one action that doesn't fall to a commit
290 -- that is already there.
291 trim :: [Committed] -> [Action] -> [Action]
292 trim _ [] = []
293 trim cs (Insert d : as) = Insert d : trim cs as
294 trim [] (_ : as) = trim [] as
295 trim (_ : cs) (a : as) = a : trim cs as
296
297 -- | A plan as a failing case should read: the version the branch starts out as,
298 -- and then what every rewrite does to the stretch under the fork point and to
299 -- the stretch above it, bottom commit first.
300 instance Show Plan where
301   show p =
302     intercalate "\n" $
303       "initial"
304       : stretch "base" (map put (plBase p))
305       ++ stretch "branch" (map put (plBranch p))
306       ++ concat
307         [ ("rewrite " ++ show i)
308           : stretch "base" (map does (stpBase s))
309           ++ stretch "branch" (map does (stpBranch s))
310         | (i, s) <- zip [1 :: Int ..] (plSteps p)
311         ]
312   where
313     stretch what [] = [" " ++ what ++ ": nothing"]
314     stretch what as = [" " ++ what ++ ":") : map (" " ++ as)
315
316     does Keep = "keep"
317     does Drop = "drop"
318     does (Rework d) = "rework into " ++ says d
319     does (Insert d) = put d
320
321     put d = "insert " ++ says d
322     says d = show (RangeDiff.messageText (cdMessage d)) ++ " / " ++ show (cdLine d)

```

./rangediff/testlib/RangeDiff/Test/Repo.hs [435 lines]

```

1 -- | A repo whose branch is rewritten over and over, the way force-pushing one
2 -- rewrites it.
3 --
4 -- What happens to the repo is not decided here: it is drawn as a
5 -- "RangeDiff.Test.Plan", which is planned in full before the repo exists.
6 -- 'withRepo' draws a plan and builds a repo holding the first version of its
7 -- branch; 'evolve' then carries out every rewrite the plan holds. An evolve
8 -- knows what each commit was rewritten into, which is what a range-diff says,
9 -- so it can hand one back.
10 module RangeDiff.Test.Repo
11   ( Repo,
12     repoDir,

```

```

13 repoRangeDiff,
14   withRepo,
15   Rewrite (..),
16   evolve,
17   Range (..),
18   range,
19   Movement (..),
20   baseMovement,
21 )
22 where
23
24 import Control.Monad (void, when)
25 import Data.IORef (IORef, atomicModifyIORef', modifyIORef', newIORef, readIORef)
26 import Data.List (find, intercalate, unsnoc)
27 import Data.List.Extra (trim)
28 import Data.Maybe (fromMaybe, isNothing)
29 import RangeDiff qualified
30 import RangeDiff.CommitSha (CommitSha, knownSha, shaText)
31 import RangeDiff.Test.Git (git, isAncestorOf)
32 import RangeDiff.Test.Plan (Action (..), Committed (..), Plan (..), Step
33   (..), plan, shrinkPlan)
34 import System.Directory (withCurrentDirectory)
35 import System.Exit (ExitCode (..))
36 import System.IO.Temp (withSystemTempDirectory)
37 import System.Process (readProcessWithExitCode)
38 import Test.QuickCheck (Property, Testable, forAllShrink, ioProperty, suchThat)
39 -- * The repo a scenario is carried out in
40
41 -- | jj's own name for a commit, which survives every rewrite of it: a reword,
42 -- an amend, and the rebase either of those sets off above it. It is how a
43 -- scenario finds a commit again however the versions since have rewritten it.
44 newtype ChangeId = ChangeId {changeIdText :: String}
45   deriving (Eq)
46
47 -- | One commit of the branch, as the repo has it.
48 data Commit = Commit
49   { cmChange :: ChangeId,
50     -- | what its file is called, which is a file no other commit ever touches
51     cmName :: String,
52     cmCommitted :: Committed
53   }
54
55 -- | What the repo holds: the commit every version grows from, how many commits
56 -- it has ever been given (which is where the next one's name comes from), and
57 -- the branch as it stands ? the commits under the fork point and the commits
58 -- above it, each bottom commit first.
59 data State = State
60   { stRoot :: CommitSha,
61     stNamed :: Int,
62     stBase, stBranch :: [Commit]
63   }
64
65 -- | A repo a scenario is being carried out in.
66 data Repo = Repo
67   { -- | where it is, so that whatever else is done with it (opening a pull
68     -- request on it, cloning it) can be done from anywhere
69     repoDir :: FilePath,
70     -- | the rewrites still to come, in order
71     rpSteps :: [Step],
72     rpState :: IORef State
73   }
74
75 -- | The repo behind a range-diff handle, which is what everything taking a
76 -- range-diff in it goes through.
77 repoRangeDiff :: Repo -> RangeDiff.Handle

```

```

78 repoRangeDiff = RangeDiff.git . repoDir
79
80 -- | Plan a scenario of between @lo@ and @hi@ rewrites, make a repo holding the
81 -- first version of its branch, and run the property in it. Everything the
82 -- property goes on to do runs with that repo as the working directory.
83 --
84 -- Only plans @fit@ takes are built, and only those are shrunk to, so a property
85 -- that can't be run on every scenario says so here rather than throwing the
86 -- repo away once it has been built. "RangeDiff.Test.Plan"'s @unambiguous@ is
87 -- the one such thing there is to ask.
88 withRepo :: (Testable prop) => (Int, Int) -> (Plan -> Bool) -> (Repo -> IO prop) ->
  Property
89 withRepo bounds fit act =
90   forAllShrink (plan bounds `suchThat` fit) (filter fit . shrinkPlan) $ \p ->
91     ioProperty $ withSystemTempDirectory "gh-post-range-diff" $ \dir ->
92       withCurrentDirectory dir $ do
93         initRepo dir
94         root <- commitRoot
95         st <- newIORef (State root 0 [] [])
96         let repo = Repo dir (plSteps p) st
97         initialize repo p
98         act repo
99
100 -- | Commit the first version of the branch: the commits the plan puts under the
101 -- fork point, and the commits it puts above it.
102 initialize :: Repo -> Plan -> IO ()
103 initialize repo p = do
104   root <- stRoot <$> readIORef (rpState repo)
105   built <- apply repo (shaText root) [] (map Insert (plBase p ++ plBranch p))
106   let (below, above) = splitAt (length (plBase p)) built
107   modifyIORef' (rpState repo) $ \st -> st {stBase = below, stBranch = above}
108
109 -- | Where the branch begins and where it ends, as the repo has it now.
110 data Range = Range
111   { -- | the commit the branch forks off, which is where its range is taken from
112     rgBase :: CommitSha,
113     -- | its tip
114     rgHead :: CommitSha
115   }
116
117 -- | The range the branch spans as the repo has it now. This is what a
118 -- range-diff is taken over, and what a push moves the two refs of a pull
119 -- request onto.
120 range :: Repo -> IO Range
121 range repo = do
122   st <- readIORef (rpState repo)
123   v <- versionOf (stBase st ++ stBranch st)
124   let tipOf = fmap (shaOf v . snd) . unsnoc
125   pure
126     Range
127       { rgBase = fromMaybe (stRoot st) (tipOf (stBase st)),
128         rgHead = fromMaybe (error "the branch has no commits") (tipOf (stBranch st))
129       }
130
131 -- | What the base under the branch did between where it was and where it is.
132 -- Only good for saying what a scenario covered.
133 data Movement
134 = -- | it is where it was: the version was pushed onto the base the one before
135   -- it was pushed onto
136   Stayed
137 | -- | it grew, so it got where it is by an ordinary push, which leaves no
138   -- force-push event to reconstruct it from
139   Advanced
140 | -- | it moved off the line it was on, which takes a force-push, and that
141   -- leaves an event behind
142   Forced

```

```

143 deriving (Eq, Show)
144
145 baseMovement :: Repo -> CommitSha -> CommitSha -> IO Movement
146 baseMovement repo was now
147   | was == now = pure Stayed
148   | otherwise = do
149     fastForward <- isAncestorOf (repoDir repo) was now
150     pure (if fastForward then Advanced else Forced)
151
152 -- | One rewrite of the branch, as it happened: where the branch was, where it
153 -- is now, and the range-diff from the one version to the other.
154 data Rewrite = Rewrite
155   { rwWas, rwNow :: Range,
156     rwRangeDiff :: [RangeDiff.Commit]
157   }
158
159 -- | Rewrite the branch into the next version of itself, once for every rewrite
160 -- the plan holds, and hand each one to @act@.
161 --
162 -- Each is handed over before the next is made, so that whatever @act@ does about
163 -- a push has only what that push left behind to go on ? the way anything
164 -- reporting on a pull request sees only the timeline as it stood at the time.
165 evolve :: Repo -> (Rewrite -> IO a) -> IO [a]
166 evolve repo act = mapM one (rpSteps repo)
167   where
168     one s = do
169       from <- range repo
170       st <- readIORef (rpState repo)
171       was <- versionOf (stBranch st)
172       base <- apply repo (shaText (stRoot st)) (stBase st) (stpBase s)
173       branch <- apply repo (fromMaybe (shaText (stRoot st)) (revOf base)) (stBranch st)
174       (stpBranch s)
175       modifyIORef' (rpState repo) $ \s' -> s' {stBase = base, stBranch = branch}
176       now <- versionOf branch
177       to <- range repo
178       act (Rewrite from to (rangeDiffOf was now))
179
180     revOf = fmap (changeIdText . cmChange . snd) . unsnoc
181
182 -- * Doing it to the repo
183
184 -- | Read a plan's actions against the stretch of branch they were planned for,
185 -- bottom commit first, and hand back the stretch they leave behind. @below@ is
186 -- the revision that stretch sits on: what a commit put in at the bottom of it
187 -- goes after.
188 --
189 -- This is "RangeDiff.Test.Plan"'s @applied@ carried out for real, and the two
190 -- have to stay in step.
191 apply :: Repo -> String -> [Commit] -> [Action] -> IO [Commit]
192 apply _ _ cs [] = pure cs
193 apply repo below cs (a : as) = case a of
194   Insert d -> do
195     c <- insert repo below d
196     above c cs
197   Keep -> next above
198   Rework d -> next $ \c rest -> do
199     rework c d
200     above c {cmCommitted = d} rest
201 -- Abandoning a commit rebases whatever sat above it onto whatever sat below
202 -- it, so the branch closes over the gap.
203 Drop -> next $ \c rest -> do
204   _ <- jj ["abandon", changeIdText (cmChange c)]
205   apply repo below rest as
206 where
207   above c rest = (c :) <$> apply repo (changeIdText (cmChange c)) rest as

```

```

208 next f = case cs of
209     [] -> error "plan acts on a commit that isn't there"
210     c : rest -> f c rest
211
212 -- | Put a commit in that wasn't there, straight above @below@.
213 insert :: Repo -> String -> Committed -> IO Commit
214 insert repo below d = do
215     name <- fresh repo
216     _ <- jj ["new", "--insert-after", below, message (cdMessage d)]
217     -- The new commit is the one jj left the working copy on, so writing the file
218     -- is all it takes to put it in the commit: the next jj command snapshots the
219     -- working copy, and the one reading the change id back is that command.
220     writeFile (file name) (contents name (cdLine d))
221     c <- ChangeId . trim <$> jj ["log", "--no-graph", "-r", "@", "-T", "change_id"]
222     pure (Commit c name d)
223
224 -- | Rewrite a commit into what the next version makes of it: a reword goes
225 -- straight onto the commit, and an amend goes through the working copy, since
226 -- checking a commit out and writing the file is the only way to change what it
227 -- holds.
228 rework :: Commit -> Committed -> IO ()
229 rework c d = do
230     when (cdMessage (cmCommitted c) /= cdMessage d) $
231         void (jj ["describe", "-r", rev, message (cdMessage d)])
232     when (cdLine (cmCommitted c) /= cdLine d) $ do
233         _ <- jj ["edit", rev]
234         writeFile (file (cmName c)) (contents (cmName c) (cdLine d))
235     where
236         rev = changeIdText (cmChange c)
237
238 -- | A name no commit in the repo has been given before, for the file the next
239 -- one commits to.
240 fresh :: Repo -> IO String
241 fresh repo = atomicModifyIORef' (rpState repo) $ \st ->
242     (st {stNamed = stNamed st + 1}, "f" ++ show (stNamed st + 1))
243
244 -- | A stretch of branch as it is at some point: the commits it is made of, and
245 -- the commit each of them is on.
246 data Version = Version
247     { vsCommits :: [Commit],
248       vsShas :: [(ChangeId, CommitSha)]
249     }
250
251 -- | Read what commits a stretch of branch is on as the repo stands.
252 versionOf :: [Commit] -> IO Version
253 versionOf [] = pure (Version [] [])
254 versionOf cs = Version cs . map entry . lines <$> jj ["log", "--no-graph", "-r"
255     , revset, "-T", template]
256     where
257         revset = intercalate " | " [changeIdText (cmChange c) | c <- cs]
258         template = "change_id ++ \" \" ++ commit_id ++ \"\\n\\n\""
259         entry l = case words l of
260             [c, sha] -> (ChangeId c, knownSha sha)
261             _ -> error ("unexpected jj log line: " ++ l)
262
263 -- | The commit one of them is on.
264 shaOf :: Version -> Commit -> CommitSha
265 shaOf v c = fromMaybe (error ("jj lost "
266     ++ cmName c)) (lookup (cmChange c) (vsShas v))
267
268 -- * The range-diff the rewrite amounts to
269
270 -- | The range-diff from the branch as it was to the branch as it is: what
271 -- happened to every commit of either version, in the order `git range-diff`
272 -- prints them, with the commit each of them is on.
273

```

```

272 -- which commit of the old version is which commit of the new one is jj's
273 -- business, and git works it out for itself, by how alike two commits' patches
274 -- are. The two agree because every commit touches a file of its own, and only
275 -- ever one line of it, so a rewritten commit is far more like what it was than
276 -- like anything else in either version.
277 rangeDiffOf :: Version -> Version -> [RangeDiff.Commit]
278 rangeDiffOf was now = go (vsCommits was) (vsCommits now) []
279   where
280     -- git walks the two versions at once, printing a commit the new version
281     -- doesn't have where the old one has it, and every commit the old version
282     -- doesn't have where the new one has it. A commit both of them have is
283     -- printed where the new version has it, which is why the old version has to
284     -- be walked past the ones already printed.
285     go os ns shown
286     | (o : rest) <- unshown, isNothing (matching o new) = dropped o : go rest ns sho
287   wn
288     | (n : ns') <- ns = case matching n old of
289       Nothing -> introduced n : go unshown ns' shown
290       Just o -> paired o n : go unshown ns' (cmChange o : shown)
291     | otherwise = []
292   where
293     unshown = dropWhile ((`elem` shown) . cmChange) os
294     matching c = find ((== cmChange c) . cmChange)
295
296     old = vsCommits was
297     new = vsCommits now
298
299     -- git takes the message on a header line from the old side of every pair it
300     -- lines up, so a reworted commit is reported under the message it is being
301     -- rewritten away from.
302     paired o n
303     | cmCommitted o == cmCommitted n = entry RangeDiff.Unchanged (shaOf now n) n
304     | otherwise = entry (RangeDiff.Updated (interdiff o n)) (shaOf now n) o
305
306     -- A dropped commit is only on the old side, so that is the side it is
307     -- reported from.
308     dropped o = entry RangeDiff.Removed (shaOf was o) o
309     introduced n = entry RangeDiff.Added (shaOf now n) n
310
311     entry change sha c = RangeDiff.Commit change sha (cdMessage (cmCommitted c))
312
313 -- | The interdiff git prints under a rewritten commit, as
314 -- 'RangeDiff.rangeDiff' should hand it back: de-indented, so its own +/- sit in
315 -- column 0. A rewrite that reworted the commit gets a hunk over the metadata
316 -- git shows a commit under, one that amended it gets a hunk over the patch, and
317 -- one that did both gets both hunks, in that order.
318 interdiff :: Commit -> Commit -> RangeDiff.Interdiff
319 interdiff a b = RangeDiff.interdiff (unlines (reworted ++ amended))
320   where
321     (was, now) = (cmCommitted a, cmCommitted b)
322     name = cmName b
323
324     -- The message sits in a section of its own, indented by four, with the
325     -- author above it and the head of the patch below ? three lines of context
326     -- either side, as a default @-U3@ diff leaves.
327     reworted
328     | cdMessage was == cdMessage now = []
329     | otherwise =
330       [ "@@ Metadata",
331         " Author: " ++ auName author ++ " <" ++ auEmail author ++ ">",
332         " ",
333         " ## Commit message ##",
334         "- " ++ RangeDiff.messageText (cdMessage was),
335         "+ " ++ RangeDiff.messageText (cdMessage now),
336         " ",

```



```

337 "##" ++ file name ++ " (new) ##",
338 "  @"
339 ]
340
341 -- Context here is the tail of the file's body, as the new patch adds it.
342 amended
343 | cdLine was == cdLine now = []
344 | otherwise =
345   ["@@ " ++ file name ++ " (new)"
346    ++ [" " ++ body name k | k <- [6 .. 8]]
347    ++ ["-" ++ cdLine was, "+" ++ cdLine now]
348
349 -- * The repo underneath
350
351 -- | The file a commit commits to. One file each, so unrelated commits can never
352 -- be mistaken for one another.
353 file :: String -> FilePath
354 file name = name ++ ".txt"
355
356 -- | The @k@th body line of a commit's file. Naming the commit it belongs to
357 -- keeps two commits' patches from resembling one another, which would let
358 -- range-diff pair a dropped commit with an unrelated one that was put in.
359 body :: String -> Int -> String
360 body name k = name ++ "-l" ++ show k
361
362 -- | A commit's file, ending on the line the version carrying it chose. The body
363 -- is long enough that rewriting that one line is a small fraction of the
364 -- commit, which keeps range-diff pairing the two versions of it (marker @!@)
365 -- instead of treating them as an unrelated drop plus an unrelated insert.
366 contents :: String -> String -> String
367 contents name l = unlines ([body name k | k <- [1 .. 8]] ++ [l])
368
369 -- | Who commits everything in the repo. The scenario needs it too: it is the
370 -- one line of commit metadata a range-diff shows around a reworded message.
371 data Author = Author
372   { auName :: String,
373     auEmail :: String
374   }
375
376 author :: Author
377 author = Author {auName = "t", auEmail = "t@t"}
378
379 -- | Run jj under the settings every repo here is built with, handing back what
380 -- it printed on stdout. jj narrates what it rewrote on stderr, which is noise
381 -- while the build goes to plan and the whole story when it doesn't, so it is
382 -- kept back until a command fails.
383 jj :: [String] -> IO String
384 jj args = do
385   (code, out, err) <- readProcessWithExitCode "jj"
386     (concatMap asOption settings ++ args) ""
387   case code of
388     ExitSuccess -> pure out
389     ExitFailure _ -> fail ("jj " ++ unwords args ++ " failed:\n" ++ err)
390 where
391   asOption (k, v) = ["--config", k ++ "=" ++ v]
392
393 -- | Everything the build leans on, settled here rather than left to the jj
394 -- config of whoever is running the tests.
395 --
396 -- Handed to every command rather than written to the repo's own config: jj
397 -- keeps repo-level config under the user's config directory, keyed by the path
398 -- of the repo, so writing it needs a home to write to ? which a sandboxed
399 -- build hasn't got ? and would leave an entry behind for every throwaway repo
400 -- the tests build. Options on the command line need no home at all, and win
401 -- over a user's config just the same.
402 settings :: [(String, String)]

```

```

402 settings =
403   [ ("user.name", auName author),
404     ("user.email", auEmail author),
405     -- Every file a commit writes is a file it means to commit.
406     ("snapshot.auto-track", "all()"),
407     -- Rewriting history is what these repos are for, so nothing in them is
408     -- history jj should be holding on to.
409     ("revset-aliases.\\"immutable_heads()\\"", "none()")
410   ]
411
412 -- | How a message is handed to jj.
413 message :: RangeDiff.CommitMessage -> String
414 message m = "--message=" ++ RangeDiff.messageText m
415
416 -- | A repo to rewrite a branch in. It is colocated, so that everything reading
417 -- it afterwards ? `git range-diff`, and the checkout the tool runs in ? is
418 -- reading an ordinary git repo.
419 initRepo :: FilePath -> IO ()
420 initRepo dir = do
421   _ <- jj ["git", "init", "--colocate", "."]
422   -- How far git abbreviates a sha is its own business, and it varies with the
423   -- size of the repo. Told not to abbreviate at all, `range-diff` prints the
424   -- full shas, which are the ones the scenario knows its commits by.
425   git dir ["config", "core.abbrev", "no"]
426
427 -- | The commit every version of the branch grows from, and the one a version
428 -- with nothing under its fork point forks off. jj made a commit to initialise
429 -- the repo on; this is that commit, given something to hold and something to
430 -- say.
431 commitRoot :: IO CommitSha
432 commitRoot = do
433   writeFile "root.txt" "root\n"
434   _ <- jj ["describe", "--message=root"]
435   knownSha . trim <$> jj ["log", "--no-graph", "-r", "@", "-T", "commit_id"]

```

./src/GhPostRangeDiff/Git.hs [74 lines]

```

1 -- | The repo the tool takes its range-diffs in, behind a handle, so that it can
2 -- be mocked in the tests. The one implementation that talks to a real repo is
3 -- 'git', which is that repo's directory and the `git` invocations it takes to
4 -- read it.
5 module GhPostRangeDiff.Git
6   ( abbrev,
7     Handle (..),
8     git,
9   )
10 where
11
12 import Data.List.Extra (trim)
13 import RangeDiff.CommitSha (CommitSha, knownSha, shaText)
14 import System.Exit (ExitCode (..))
15 import System.Process (callProcess, readProcess, readProcessWithExitCode)
16
17 -- | A sha cut down to the seven digits people read commits by. Only for
18 -- showing; it is git's own default abbreviation, not necessarily unambiguous.
19 abbrev :: CommitSha -> String
20 abbrev = take 7 . shaText
21
22 -- | Everything the tool does with the repo it takes its range-diffs in.
23 data Handle = Handle
24   { -- | Bring the branch named here, and every commit listed, in from origin,
25     -- and hand back the commit that branch is on. Nothing else here can be
26     -- asked about a commit this has not brought in first.
27     fetch :: String -> [CommitSha] -> IO CommitSha,
28     -- | Does @a@ come before @b@ on the same line of history?
29     isAncestorOf :: CommitSha -> CommitSha -> IO Bool,
30     -- | The best common ancestor of @a@ and @b@.

```

```

31 mergeBase :: CommitSha -> CommitSha -> IO CommitSha
32 }
33
34 -- | The repo in @dir@, as the `git` CLI reaches it.
35 git :: FilePath -> Handle
36 git dir =
37   Handle
38     { fetch = gitFetch dir,
39       isAncestorOf = gitIsAncestorOf dir,
40       mergeBase = gitMergeBase dir
41     }
42
43 -- Run a git command in the repo in @dir@, and hand back what it printed on
44 -- stdout.
45 command :: FilePath -> [String] -> IO String
46 command dir args = readProcess "git" (at dir args) ""
47
48 -- Aim a git command at the repo in @dir@. Every invocation here is aimed this
49 -- way, so which directory the process happens to be in never comes into it.
50 at :: FilePath -> [String] -> [String]
51 at dir args = "-C" : dir : args
52
53 -- Fetch the branch and the commits, then read back where the branch ended up.
54 --
55 -- refs/rd/base is a scratch ref holding the fetched branch tip. We need it
56 -- because without a destination the tip only lands in FETCH_HEAD, which this
57 -- same fetch also fills with every commit asked for, so we couldn't pick the
58 -- tip back out to rev-parse afterwards.
59 gitFetch :: FilePath -> String -> [CommitSha] -> IO CommitSha
60 gitFetch dir base commits = do
61   callProcess "git" (at dir (["fetch", "--quiet", "origin", base ++ ":refs/rd/base"
62     ] ++ map shaText commits))
63   revParse dir "refs/rd/base"
64
65 gitIsAncestorOf :: FilePath -> CommitSha -> CommitSha -> IO Bool
66 gitIsAncestorOf dir a b = do
67   (code, _, _) <- readProcessWithExitCode "git" (at dir ["merge-base", "--is-ancestor"
68     , shaText a, shaText b]) ""
69   pure (code == ExitSuccess)
70
71 gitMergeBase :: FilePath -> CommitSha -> CommitSha -> IO CommitSha
72 gitMergeBase dir a b = knownSha . trim <$> command dir ["merge-base"
73   , shaText a, shaText b]
74
75 -- The commit a revision names.
76 revParse :: FilePath -> String -> IO CommitSha
77 revParse dir rev = knownSha . trim <$> command dir ["rev-parse", rev]

```

./src/GhPostRangeDiff/GitHub.hs [128 lines]

```

1 -- | The pull request the tool reports on, behind a handle, so that it can
2 -- be mocked in the tests. The one implementation that talks to the real
3 -- thing is 'gh', with the `gh` invocations and timeline parsing it needs.
4 module GhPostRangeDiff.GitHub
5   ( Ref (...),
6     Ev (...),
7     Handle (...),
8     gh,
9   )
10 where
11
12 import Data.List.Extra (trim)
13 import RangeDiff.CommitSha (CommitSha, commitSha)
14 import System.Process (callProcess, readProcess)
15
16 -- | Which of the pull request's two branches a force-push moved: the branch
17 -- under review, or the branch it is targeting.

```

```

18 data Ref = Head | Base
19 deriving (Eq, Show)
20
21 -- One force-push: the ref it moved, its before-oid and its after-oid. GitHub
22 -- reports both oids in full, so they read straight back as shas.
23 data Ev = Ev
24   { evRef :: Ref,
25     evBefore, evAfter :: CommitSha
26   }
27
28 -- | Everything the tool does with one pull request.
29 data Handle = Handle
30   { -- | the force-push timeline events, oldest first
31     timeline :: IO [Ev],
32     -- | the branch the pull request is targeting
33     baseRef :: IO String,
34     -- | every comment body on it, concatenated
35     comments :: IO String,
36     postComment :: String -> IO ()
37   }
38
39 -- | The real pull request numbered @pr@, as the `gh` CLI reaches it.
40 gh :: String -> Handle
41 gh pr =
42   Handle
43     { timeline = ghTimeline pr,
44       baseRef = ghBaseRef pr,
45       comments = ghComments pr,
46       postComment = ghPostComment pr
47     }
48
49 -- The query asks for the two force-push events and nothing else, so any other
50 -- event type is not something we know how to read.
51 parse :: String -> Ev
52 parse l = case words l of
53   (t : b : c : _)
54     | Just r <- ref t,
55       Just before <- commitSha b,
56       Just after <- commitSha c ->
57     Ev r before after
58   _ -> error ("unexpected timeline line: " ++ l)
59 where
60   ref "HeadRefForcePushedEvent" = Just Head
61   ref "BaseRefForcePushedEvent" = Just Base
62   ref _ = Nothing
63
64 query :: String
65 query =
66   unlines
67     [ "query($o: String!, $r: String!, $n: Int!) {",
68       "  repository(owner: $o, name: $r) {",
69         "    pullRequest(number: $n) {",
70           "      timelineItems(",
71             "        last: 250",
72             "        itemTypes: [HEAD_REF_FORCE_PUSHED_EVENT, BASE_REF_FORCE_PUSHED_EVENT]",
73           "      ) {",
74             "        nodes {",
75               "          __typename",
76
77             "... on HeadRefForcePushedEvent { beforeCommit { oid } afterCommit { oid } }",
78
79             "... on BaseRefForcePushedEvent { beforeCommit { oid } afterCommit { oid } }",
80
81             "        }",
82             "      }",
83     ]

```

```

80     " }",
81     " }",
82     "}"
83 ]
84
85 -- Run a `gh` command and hand back what it printed on stdout.
86 ghRead :: [String] -> IO String
87 ghRead args = readProcess "gh" args ""
88
89 -- The repository the `gh` CLI is pointed at, as (owner, name).
90 ownerRepo :: IO (String, String)
91 ownerRepo = do
92   nameWithOwner <- trim <$> ghRead ["repo", "view", "--json", "nameWithOwner", "-q",
93     ".nameWithOwner"]
94   case break (== '/') nameWithOwner of
95     (owner, _ : repo) -> pure (owner, repo)
96     _ -> error ("unexpected repository name: " ++ nameWithOwner)
97
98 -- The PR's force-push timeline events, oldest first.
99 ghTimeline :: String -> IO [Ev]
100 ghTimeline pr = do
101   (owner, repo) <- ownerRepo
102   raw <-
103     ghRead
104       [ "api",
105         "graphql",
106         "-f",
107         "query=" ++ query,
108         "-F",
109         "o=" ++ owner,
110         "-F",
111         "r=" ++ repo,
112         "-F",
113         "n=" ++ pr,
114         "--jq",
115         ".data.repository.pullRequest.timelineItems.nodes[]"
116         ++ " | \"\\(.__typename) \\(\\.beforeCommit.oid) \\(\\.afterCommit.oid)\""
117       ]
118   pure (map parse (lines raw))
119
120 -- The branch the PR is targeting.
121 ghBaseRef :: String -> IO String
122 ghBaseRef pr = trim <$> ghRead ["pr", "view", pr, "--json", "baseRefName", "-q",
123   ".baseRefName"]
124
125 -- Every comment body on the PR, concatenated.
126 ghComments :: String -> IO String
127 ghComments pr = ghRead ["pr", "view", pr, "--json", "comments", "-q",
128   ".comments[].body"]
129
130 ghPostComment :: String -> String -> IO ()
131 ghPostComment pr body = callProcess "gh" ["pr", "comment", pr, "--body", body]

```

./src/GhPostRangeDiff/Render.hs [46 lines]

```

1 -- | Turn a parsed range-diff into a Markdown list for a GitHub PR comment.
2 --
3 -- Each commit becomes a list item with a status emoji, an auto-linked commit
4 -- sha, and ? for a changed commit ? its interdiff shown in-place as a fenced
5 -- diff block.
6 module GhPostRangeDiff.Render (format) where
7
8 import Data.List (group, intercalate)
9 import RangeDiff (Change (..), Commit (..), interdiffText, messageText)
10 import RangeDiff.CommitSha (shaText)
11
12 -- | The longest run of consecutive backticks in a string. Used to size a code

```

```

13 -- fence so it can't be closed early by backticks in the content.
14 maxBacktickRun :: String -> Int
15 maxBacktickRun s = maximum (0 : [length g | g@('`' : _) <- group s])
16
17 render :: Commit -> String
18 render (Commit change sha subj) =
19   -- The sha is left bare (not in a `code` span) so GitHub auto-links it to
20   -- the commit.
21   tag ++ " " ++ shaText sha ++ " " ++ messageText subj ++ details
22   where
23     tag = case change of
24       Added -> "\128994 **Added**" -- ?
25       Removed -> "\128308 **Removed**" -- ?
26       Updated _ -> "\128992 **Updated**" -- ?
27       Unchanged -> "\9898 **Unchanged**" -- ?
28       -- Only a changed commit carries an interdiff worth showing.
29     details = case change of
30       Updated patch -> fenced (interdiffText patch)
31       _ -> ""
32
33 -- | Show a patch as a fenced diff block. The fence is longer than any backtick
34 -- run in the patch, so a line like `` inside the interdiff can't close the
35 -- block early. An empty patch gets no block at all. The closing fence needs no
36 -- newline in front of it: a 'RangeDiff.Interdiff' ends on a
37 -- line boundary.
38 fenced :: String -> String
39 fenced "" = ""
40 fenced patch = "\n\n" ++ fence ++ "diff\n" ++ patch ++ fence
41   where
42     fence = replicate (max 3 (maxBacktickRun patch + 1)) '`'
43
44 -- | Render the commits as a Markdown list, one item per commit.
45 format :: [Commit] -> String
46 format = intercalate "\n\n" . map render

```

./src/GhPostRangeDiff/Run.hs [96 lines]

```

1 -- | Reporting a push on a PR, as the CLI drives it.
2 module GhPostRangeDiff.Run (Reported (..), run, Bases (..), bases) where
3
4 import Control.Monad.Extra (andM, findM, notM)
5 import Data.List (isInfixOf, nub)
6 import GhPostRangeDiff.Git (abbrev)
7 import GhPostRangeDiff.Git qualified as Git
8 import GhPostRangeDiff.GitHub (Ev (..), Ref (..))
9 import GhPostRangeDiff.GitHub qualified as GitHub
10 import GhPostRangeDiff.Render (format)
11 import RangeDiff qualified
12 import RangeDiff.CommitSha (CommitSha, shaText)
13
14 -- Hidden marker identifying this exact push (before..after). Lets us run the
15 -- program multiple times without duplicating comments.
16 marker :: CommitSha -> CommitSha -> String
17 marker oldHead newHead =
18   "<!-- gh-post-range-diff " ++ shaText oldHead ++ ".." ++ shaText newHead ++ " -->"
19
20 -- | What reporting on a push came to: the comment went up, or the marker of
21 -- that push was on the PR already, so a previous run had said it all.
22 data Reported = Posted | AlreadyReported
23   deriving (Eq, Show)
24
25 -- Report on the push oldHead..newHead: post what there is to say about it as a
26 -- PR comment, under the marker of that push, unless the marker says we already
27 -- have.
28 run :: Git.Handle -> RangeDiff.Handle -> GitHub.Handle -> CommitSha -> CommitSha -> IO
   Reported
29 run repo rd pr oldHead newHead = do

```

```

30 posted <- GitHub.comments pr
31 if marker oldHead newHead `isInfixOf` posted
32 then pure AlreadyReported
33 else do
34   body <- comment repo rd pr oldHead newHead
35   GitHub.postComment pr (marker oldHead newHead ++ "\n" ++ body)
36   pure Posted
37
38 -- | Report on the push oldHead..newHead: a header naming both ends of it, and
39 -- the range-diff between the version of the branch it replaced and the one it
40 -- left behind.
41 comment :: Git.Handle -> RangeDiff.Handle -> GitHub.Handle -> CommitSha -> CommitSha
42         -> IO String
43 comment repo rd pr oldHead newHead = do
44   Bases b1 b2 <- bases repo pr oldHead newHead
45   commits <- RangeDiff.rangeDiff rd b1 oldHead b2 newHead
46   let header = "### Range-diff for push " ++ abbrev oldHead ++ " ? " ++ abbrev newHead
47   pure (header ++ "\n\n" ++ format commits)
48
49 -- | Where each of the two versions of the branch forks off, which is where the
50 -- range-diff between them is taken from.
51 data Bases = Bases
52   { -- | the base the version being replaced was on
53     basesOld :: CommitSha,
54     -- | the base the version left behind is on
55     basesNew :: CommitSha
56   }
57 deriving (Eq, Show)
58
59 -- | Work out those two for the push oldHead..newHead, asking the pull request
60 -- where its base branch is and what that branch has been through, and bringing
61 -- everything that takes into the repo. Nothing else here has to have fetched
62 -- anything first: this is what puts the commits within reach, the heads
63 -- included, so that the range-diff taken over them afterwards can read them.
64 --
65 -- Where the base branch has only ever been fast-forwarded, it leaves no
66 -- force-push events behind, and where it is now is enough: each version forks
67 -- off it wherever it merges back.
68 --
69 -- Where it has been force-pushed, the version being replaced may have been on a
70 -- base tip that is no longer on the branch, and merging against the branch as it
71 -- stands would take the range-diff from the wrong place. So look for the most
72 -- recent recorded tip that the old head was on and the branch has since lost ?
73 -- that is the base it was pushed against ? and fall back to the merge base
74 -- where there is none.
75 bases :: Git.Handle -> GitHub.Handle -> CommitSha -> CommitSha -> IO Bases
76 bases repo pr oldHead newHead = do
77   base <- GitHub.baseRef pr
78   -- Every recorded base tip, in chronological order, for base reconstruction.
79   recorded <-
80     concatMap (\e -> [evBefore e, evAfter e]) . filter ((== Base) . evRef)
81     <$> GitHub.timeline pr
82
83   -- Fetch current base tip, both heads, and every historical base oid.
84   newBaseTip <- Git.fetch repo base (nub (oldHead : newHead : recorded))
85   if null recorded
86   then Bases <$> mergeBase newBaseTip oldHead <*> mergeBase newBaseTip newHead
87   else do
88     lostAncestorOfOldHead <-
89       findM
90       (\tl -> andM [tl `isAncestorOf` oldHead, notM (tl `isAncestorOf` newBaseTip)])
91       (reverse recorded)
92     b1 <- maybe (mergeBase newBaseTip oldHead) pure lostAncestorOfOldHead
93     pure (Bases b1 newBaseTip)

```

```

94 where
95   isAncestorOf = Git.isAncestorOf repo
96   mergeBase = Git.mergeBase repo

```

./test/GhPostRangeDiff/BasesSpec.hs [258 lines]

```

1  -- | Working out where the two versions of a branch fork off, over a base
2  -- branch that keeps moving under them.
3  --
4  -- A scenario here is a list of versions of the repo: the base branch as it
5  -- stands, and the one commit that is the feature branch, sitting somewhere
6  -- along it. Each version is pushed in turn, and the answer is known ahead of
7  -- the asking ? a version forks off the base branch wherever its commit sits ?
8  -- so what is being asked is whether the tool can work that back out from the
9  -- pull request's timeline, however far the base branch has moved since.
10 --
11 -- There are two ways a base branch moves, and a property each. It is rewritten:
12 -- commits are taken out of it, put on the end of it, or it is reset to a version
13 -- it had, and the branch is rebased onto its tip every time. Or it is a trunk:
14 -- commits only ever land on the end of it, and the branch forks off somewhere
15 -- along it and catches up in its own time.
16 module GhPostRangeDiff.BasesSpec (spec) where
17
18 import GhPostRangeDiff.FakeGit (Change (..), ChangeNo (..))
19 import GhPostRangeDiff.FakeGit qualified as FakeGit
20 import GhPostRangeDiff.FakeGitHub qualified as FakeGitHub
21 import GhPostRangeDiff.Git (abbrev)
22 import GhPostRangeDiff.GitHub (Ev (..))
23 import GhPostRangeDiff.Run (Bases (..), bases)
24 import RangeDiff.CommitSha (CommitSha)
25 import Test.Hspec
26 import Test.Hspec.QuickCheck (modifyMaxSuccess)
27 import Data.List (isPrefixOf, tails)
28 import Test.QuickCheck
29
30 -- * A version of the repo
31
32 -- | The base branch's commits, bottom-most first, and how many of them the
33 -- feature commit sits on. It carries the same change every time ? the branch is
34 -- one commit, rebased over and over, never rewritten into anything else ? so
35 -- where it sits is the whole of what tells two versions of it apart.
36 data Version = Version
37   { verBase :: [ChangeNo],
38     verForkedAt :: Int
39   }
40 deriving (Eq)
41
42 -- | A version as a counterexample reads it: the base branch by its change
43 -- numbers, and how far along it the feature branch forked off.
44 instance Show Version where
45   show v = show [n | ChangeNo n <- verBase v] ++ " forked at " ++ show (verForkedAt v)
46
47 -- | The base branch as a change, its top-most commit in hand and every commit
48 -- under it named through its parents.
49 chain :: [ChangeNo] -> Change
50 chain = foldl1 (\parent c -> c {changeParent = Just parent}) . map (`Change` Nothing)
51
52 -- | The feature commit, on however much of the base branch it forked off.
53 feature :: Version -> Change
54 feature v = Change (ChangeNo 0) (Just (chain (take (verForkedAt v) (verBase v))))
55
56 -- | Where the base branch is, which is what the pull request targets.
57 baseTip :: Version -> CommitSha
58 baseTip = sha . chain . verBase
59
60 -- | Where the feature branch is, which is what is under review.
61 headTip :: Version -> CommitSha

```



```

62 headTip = sha . feature
63
64 -- | Where the feature branch forks off, which is the answer being asked for.
65 forkPoint :: Version -> CommitSha
66 forkPoint v = sha (chain (take (verForkedAt v) (verBase v)))
67
68 sha :: Change -> CommitSha
69 sha = FakeGit.cmSha . FakeGit.commit
70
71 -- | Every commit a version has: the feature commit, and the whole base branch,
72 -- which reaches past the fork point where the branch hasn't caught up with it.
73 commits :: Version -> [FakeGit.Commit]
74 commits v = under (feature v) ++ under (chain (verBase v))
75 where
76   under ch = FakeGit.commit ch : maybe [] under (changeParent ch)
77
78 -- * A base branch being rewritten
79
80 -- | How a base branch being rewritten moves from one version to the next.
81 -- These three are all of it, and the branch is rebased onto the tip after each.
82 data Op
83 = -- | take the commit at this position out of the base branch
84   Remove Int
85   | -- | put a new commit on the end of it
86   Add
87   | -- | put it back at a version it has already been at
88   Reset [ChangeNo]
89
90 -- | Start the base branch off and rewrite it, with the feature branch on its
91 -- tip throughout: a version of a rewritten base branch is only ever pushed
92 -- rebased onto it.
93 rewritten :: Gen [Version]
94 rewritten = do
95   n <- movements
96   width <- choose (1, 4)
97   map onTip <$> go n (width + 1) [map ChangeNo [1 .. width]]
98   where
99     onTip base = Version base (length base)
100
101     go 0 _ history = pure (reverse history)
102     go i next history@(cur : older) = do
103       op <- operation cur older
104       let now = apply next op cur
105       go (i - 1) (if length now > length cur then next + 1 else next) (now : history)
106     go _ _ [] = error "a scenario always has a version to move on from"
107
108 -- | What the base branch can be made to do next: put a commit on the end
109 -- always, take one out where it has one to spare, and reset back where it has
110 -- somewhere to go back to.
111 --
112 -- The commit at the bottom is never taken out. Every version of the repo grows
113 -- from the one commit the repo starts at, as a real one does, and a base branch
114 -- rewritten out from under that commit would share no history at all with the
115 -- branch under review ? which is not a rewrite, it is a different repo.
116 operation :: [ChangeNo] -> [[ChangeNo]] -> Gen Op
117 operation cur older =
118   frequency $
119     [(3, pure Add)]
120     ++ [(2, Remove <$> choose (1, length cur - 1)) | length cur > 1]
121     ++ [(1, Reset <$> elements older) | not (null older)]
122
123 -- | Carry an operation out, with @next@ the change number to give a commit that
124 -- hasn't been made before.
125 apply :: Int -> Op -> [ChangeNo] -> [ChangeNo]
126 apply next op cur = case op of
127   Add -> cur ++ [ChangeNo next]

```

```

128 Remove i -> take i cur ++ drop (i + 1) cur
129 Reset base -> base
130
131 -- * A base branch that is a trunk
132
133 -- | Grow a base branch the way a trunk grows ? commits landing on the end of
134 -- it, and nothing ever coming off ? with the feature branch forked off
135 -- somewhere along it.
136 --
137 -- Each version lands some commits, none of them sometimes, and moves the fork
138 -- point forward, by nothing at all or as far as the branch now reaches. Neither
139 -- ever goes back: that is what makes it a trunk, and what leaves the branch
140 -- forked off behind the tip until it catches up.
141 trunk :: Gen [Version]
142 trunk = do
143   n <- movements
144   width <- choose (1, 3)
145   start <- Version (upTo width) <$> choose (1, width)
146   go n start
147   where
148     go 0 v = pure [v]
149     go i v = do
150       landed <- choose (0, 2)
151       let base = upTo (length (verBase v) + landed)
152       caughtUpTo <- choose (verForkedAt v, length base)
153       (v :) <$> go (i - 1) (Version base caughtUpTo)
154
155   upTo n = map ChangeNo [1 .. n]
156
157 -- | How many times the base branch moves, settled before any of it is drawn.
158 -- Mostly the once: a base branch that moved once under a branch is the ordinary
159 -- case, and the answer has to be right there before it is worth asking what a
160 -- longer run of moves does.
161 movements :: Gen Int
162 movements = frequency [(6, pure 1), (3, choose (2, 4)), (1, choose (5, 8))]
163
164 -- * Movements that once went wrong
165
166 -- | Every move the base branch makes over a scenario: where it was, and where
167 -- it went.
168 moves :: [Version] -> [[ChangeNo], [ChangeNo]]
169 moves versions = zip bases' (drop 1 bases')
170   where
171     bases' = map verBase versions
172
173 -- | The base branch force-pushed back onto a commit it already held, which is
174 -- to say every commit it is on now, it was on before.
175 --
176 -- Its commits used to be reported as the pull request's:
177 -- https://github.com/svenvanheugten/gh-post-range-diff/pull/58
178 rewind :: [Version] -> Bool
179 rewind = any (\(was, now) -> now `isPrefixOf` was && length now < length was) . moves
180
181 -- | The base branch back on a version it had left, having been somewhere else
182 -- in between, so that one tip of it is recorded twice over.
183 --
184 -- The commit it came back onto used to be reported as removed from the pull
185 -- request: https://github.com/svenvanheugten/gh-post-range-diff/pull/60
186 returned :: [Version] -> Bool
187 returned versions = or [now `elem` drop 1
188   later | (now : later) <- tails (map verBase versions)]
189
190 -- | The base branch force-pushed, and then advanced twice by an ordinary push,
191 -- which leaves the tip it was force-pushed onto two commits behind.
192 --
193 -- Its commits used to be reported as the pull request's:

```

```

193 -- https://github.com/svenvanheugten/gh-post-range-diff/pull/56
194 forcePushedThenAdvancedTwice :: [Version] -> Bool
195 forcePushedThenAdvancedTwice versions = any forced (tails (map verBase versions))
196   where
197     forced (a : b : c : d : _) = not (a `isPrefixOf` b) && advanced b c && advanced c
198       d
199     forced _ = False
200     advanced was now = was `isPrefixOf` now && length now > length was
201 -- * The properties
202
203 spec :: Spec
204 spec =
205   describe "bases" $
206     -- Nothing here builds a repo or runs a process, so a scenario costs almost
207     -- nothing and there is no reason to be sparing with them.
208     modifyMaxSuccess (const 10_000) $ do
209       it "finds where each version forked off a base branch being rewritten" $
210         -- Every movement that has ever gone wrong has to keep being drawn: a
211         -- generator that stopped making them would leave the property passing
212         -- over ground the bugs were never on, and the bugs are what it is here
213         -- for.
214         checkCoverage $
215           forAllShrink rewritten shorter $ \versions ->
216             cover 15 (rewound versions) "rewound onto a commit it held" $
217               cover 10 (returned versions) "back on a version it had left" $
218                 cover 3 (forcePushedThenAdvancedTwice versions)
219             "force-pushed, then advanced twice" $
220               reported versions
221       it "finds where each version forked off a trunk it catches up with" $
222         forAllShrink trunk shorter $ \versions ->
223           tabulate "versions" [show (length versions)] (reported versions)
224 -- | Only ever run less of a scenario: how the base branch got where it is, is
225 -- the whole of what is being tested, so a version can't be lifted out of the
226 -- middle of it, but stopping short of one is another scenario in its own right.
227 shorter :: [Version] -> [[Version]]
228 shorter versions = [take k versions | k <- [2 .. length versions - 1]]
229
230 -- | Push every version of the branch in turn and ask, of each push, where the
231 -- two versions it is between fork off.
232 reported :: [Version] -> Property
233 reported [] = property Discard
234 reported (first : rest) = ioProperty $ do
235   origin <- FakeGit.new
236   FakeGit.store origin (concatMap commits (first : rest))
237   pr <- FakeGitHub.newPullRequest origin (baseTip first) (headTip first)
238   conjoin <$> traverse (push origin pr) (zip (first : rest) rest)
239
240 -- | Push the version of the branch, then work out where it and the version it
241 -- replaced fork off, in a repo that has fetched nothing yet: whatever that
242 -- takes, it has to go and get.
243 push :: FakeGit.Repo -> FakeGitHub.PullRequest -> (Version, Version) -> IO Property
244 push origin pr (was, now) = do
245   pushed <- FakeGitHub.push pr (baseTip now) (headTip now)
246   case pushed of
247     -- The base branch moved and the feature branch came out where it already
248     -- was, so no push happened and nothing fired. It still counts for
249     -- something: the base branch has moved, and the next push is worked out
250     -- through where it has been.
251     Nothing -> pure (property True)
252     Just ev -> do
253       checkout <- FakeGit.clone origin
254       found <- bases (FakeGit.handle checkout) (FakeGitHub
255         .handle pr) (evBefore ev) (evAfter ev)
256       pure $

```

```

256 counterexample ("pushing" ++ show now ++ " over " ++ show was) $
257   (abbrev (basesOld found), abbrev (basesNew found))
258   === (abbrev (forkPoint was), abbrev (forkPoint now))

```

./test/GhPostRangeDiff/FakeGit.hs [156 lines]

```

1 -- | A git repo with nothing underneath it: no directory, no processes, no
2 -- files. All it holds is commits ? each one naming its parent ? and the
3 -- branches pointing at them. That is enough to answer everything a
4 -- 'Git.Handle' is asked, since none of it is about what a commit holds, only
5 -- about how the commits relate.
6 --
7 -- The one thing a repo can't answer for is a commit it hasn't got: those are
8 -- errors here, as they are for git. That is what keeps the tool honest about
9 -- fetching what it is going to look at.
10 module GhPostRangeDiff.FakeGit
11   ( ChangeNo (..),
12     Change (..),
13     Commit (..),
14     commit,
15     Repo,
16     new,
17     clone,
18     store,
19     place,
20     handle,
21   )
22 where
23
24 import Data.Bits (shiftR, xor, (.&..))
25 import Data.Char (intToDigit, ord)
26 import Data.IORef (IORef, modifyIORef', newIORef, readIORef)
27 import Data.Map.Strict (Map)
28 import Data.Map.Strict qualified as Map
29 import Data.Maybe (fromMaybe)
30 import Data.Word (Word64)
31 import GhPostRangeDiff.Git qualified as Git
32 import RangeDiff.CommitSha (CommitSha, knownSha, shaText)
33
34 -- * The commits
35
36 -- | What a commit is, before it is one: the change it carries, and the change
37 -- it sits on. A change number is the only thing that tells two commits apart
38 -- here ? nothing else about a commit is ever asked of a 'Git.Handle' ? so a
39 -- chain of them is the whole of a history.
40 newtype ChangeNo = ChangeNo Int deriving (Eq, Ord, Show)
41
42 data Change = Change
43   { changeNo :: ChangeNo,
44     changeParent :: Maybe Change
45   }
46   deriving (Eq, Show)
47
48 -- | A commit as these repos have it: the sha it is known by, and what it sits
49 -- on.
50 data Commit = Commit
51   { cmSha :: CommitSha,
52     cmParent :: Maybe CommitSha
53   }
54   deriving (Eq, Show)
55
56 -- | The commit a change comes out as. Its sha stands for everything under it
57 -- as well as the change itself, the way a git sha does: rewriting a commit
58 -- rewrites every commit above it, so a change carried on a different history
59 -- is a different commit.
60 commit :: Change -> Commit
61 commit ch = Commit (shaOf ch) (cmSha . commit <$> changeParent ch)

```

```

62
63 -- Make up a sha for a change and everything under it, digested down to the
64 -- forty hex digits a sha is read as. Two commits differing anywhere in that
65 -- chain come out differently, which is as much as anything here needs.
66 shaOf :: Change -> CommitSha
67 shaOf ch = knownSha (take 40 (concatMap digest [0 .. 4 :: Int]))
68   where
69     seed = show (numbers ch)
70     numbers c = changeNo c : maybe [] numbers (changeParent c)
71     digest i = hex (foldl step (0x811c9dc5 + fromIntegral i) (map ord seed))
72     step h c = (h `xor` fromIntegral c) * 0x100000001b3 :: Word64
73     hex w = [intToDigit (fromIntegral ((w `shiftR` (4 * b)) .&. 15)) | b <- [15, 14
        .. 0]]
74
75 -- * The repo
76
77 -- | What a repo is: the commits it holds, the branches it has, and the repo it
78 -- fetches from, where it has one.
79 data Repo = Repo
80   { rpOrigin :: Maybe Repo,
81     rpCommits :: IORef (Map CommitSha Commit),
82     rpBranches :: IORef (Map String CommitSha)
83   }
84
85 -- | A repo of its own, with nothing in it and nowhere to fetch from.
86 new :: IO Repo
87 new = Repo Nothing <$> newIORef Map.empty <*> newIORef Map.empty
88
89 -- | A repo that fetches from another one. It starts out empty, which is the
90 -- point of it: whatever the tool means to look at, it has to go and get.
91 clone :: Repo -> IO Repo
92 clone origin = Repo (Just origin) <$> newIORef Map.empty <*> newIORef Map.empty
93
94 -- | Put commits in the repo, the way pushing a branch puts every commit under
95 -- it there.
96 store :: Repo -> [Commit] -> IO ()
97 store repo cs =
98   modifyIORef' (rpCommits repo) (Map.union (Map.fromList [(cmSha c, c) | c <- cs]))
99
100 -- | Put a branch on a commit.
101 place :: Repo -> String -> CommitSha -> IO ()
102 place repo b s = modifyIORef' (rpBranches repo) (Map.insert b s)
103
104 -- | Everything the tool does with a repo, answered by walking the commits.
105 handle :: Repo -> Git.Handle
106 handle repo =
107   Git.Handle
108     { Git.fetch = fetch repo,
109       Git.isAncestorOf = \a b -> asking (\m -> isAncestorOf m a b),
110       Git.mergeBase = \a b -> asking (\m -> mergeBase m a b)
111     }
112   where
113     asking f = f <$> readIORef (rpCommits repo)
114
115 -- | Bring a branch in from origin, and the commits named alongside it, and
116 -- hand back where that branch is. What is under them comes too: a fetch brings
117 -- the history of whatever it fetches.
118 fetch :: Repo -> String -> [CommitSha] -> IO CommitSha
119 fetch repo b ss = do
120   origin <- maybe (fail "fetch: the repo has no origin") pure (rpOrigin repo)
121   m <- readIORef (rpCommits origin)
122   branches <- readIORef (rpBranches origin)
123   let tip = fromMaybe (error ("origin has no branch " ++ b)) (Map.lookup b branches)
124   store repo (concatMap (ancestry m) (tip : ss))
125   pure tip
126

```

127 -- * Reading the commits

```

128
129 -- | The commit a sha names. A repo can only answer for what it holds, so a sha
130 -- it hasn't got is an error rather than a 'Nothing' to be swallowed.
131 lookedUp :: Map CommitSha Commit -> CommitSha -> Commit
132 lookedUp m s = fromMaybe (error ("no such commit: " ++ shaText s)) (Map.lookup s m)
133
134 -- | A commit and every commit under it, nearest first.
135 ancestry :: Map CommitSha Commit -> CommitSha -> [Commit]
136 ancestry m s = c : maybe [] (ancestry m) (cmParent c)
137   where
138     c = lookedUp m s
139
140 -- | Whether @@ is @b@ or a commit under it. Both have to be commits the repo
141 -- holds: @@ is looked up rather than just compared against, so asking about
142 -- one that was never fetched is an error and not a no.
143 isAncestorOf :: Map CommitSha Commit -> CommitSha -> CommitSha -> Bool
144 isAncestorOf m a b = cmSha (lookedUp m a) `elem` shas (ancestry m b)
145
146 -- | The nearest commit both of them are on. History here is a tree ? every
147 -- commit has one parent at most ? so the first commit under @b@ that is also
148 -- under @@ is the nearest such.
149 mergeBase :: Map CommitSha Commit -> CommitSha -> CommitSha -> CommitSha
150 mergeBase m a b = case filter (`elem` shas (ancestry m a)) (shas (ancestry m b)) of
151   (s : _) -> s
152   -- git exits non-zero here, which the tool doesn't survive either.
153   [] -> error "no merge base"
154
155 shas :: [Commit] -> [CommitSha]
156 shas = map cmSha

```

./test/GhPostRangeDiff/FakeGitHub.hs [105 lines]

```

1 -- | A fake pull request on a repo.
2 module GhPostRangeDiff.FakeGitHub
3   ( PullRequest,
4     newPullRequest,
5     origin,
6     handle,
7     push,
8     comments,
9   )
10 where
11
12 import Control.Monad (unless)
13 import Data.IORef (IORef, modifyIORef', newIORef, readIORef, writeIORef)
14 import GhPostRangeDiff.FakeGit qualified as FakeGit
15 import GhPostRangeDiff.Git qualified as Git
16 import GhPostRangeDiff.GitHub qualified as GitHub
17 import RangeDiff.CommitSha (CommitSha)
18
19 data PullRequest = PullRequest
20   { -- | the repo the pull request is on, which it owns: nothing else places a
21     -- branch on it, and every checkout of it is a clone
22     prOrigin :: FakeGit.Repo,
23     -- | where each of the two branches is now, which is what a push moves and
24     -- what the next one is recorded against
25     prBaseTip, prHeadTip :: IORef CommitSha,
26     prTimeline :: IORef [GitHub.Ev],
27     prComments :: IORef [String]
28   }
29
30 -- | The branches the pull request is between: the one it targets, and the one
31 -- under review. Names of our choosing, so keep them clear of whatever else the
32 -- repo puts branches under.
33 branch :: GitHub.Ref -> String
34 branch GitHub.Base = "pr-base"

```

```

35 branch GitHub.Head = "pr-head"
36
37 -- | A pull request just opened on the repo in @dir@: where the branch it
38 -- targets is, and where the branch under review is. Both branches were pushed
39 -- before the pull request could exist, and neither of those pushes is anything
40 -- it records.
41 newPullRequest :: FakeGit.Repo -> CommitSha -> CommitSha -> IO PullRequest
42 newPullRequest repo base head' = do
43   pr <-
44     PullRequest repo
45       <$> newIORef base
46       <*> newIORef head'
47       <*> newIORef []
48       <*> newIORef []
49   place pr GitHub.Base base
50   place pr GitHub.Head head'
51   pure pr
52
53 -- | The repo it is on, which is what a checkout of it clones from.
54 origin :: PullRequest -> FakeGit.Repo
55 origin = prOrigin
56
57 -- Where one of the two branches is now.
58 tip :: PullRequest -> GitHub.Ref -> IORef CommitSha
59 tip pr GitHub.Base = prBaseTip pr
60 tip pr GitHub.Head = prHeadTip pr
61
62 -- Put one of the two branches on a sha.
63 place :: PullRequest -> GitHub.Ref -> CommitSha -> IO ()
64 place pr ref sha = FakeGit.place (prOrigin pr) (branch ref) sha
65
66 -- | The 'GitHub.Handle' onto it, standing in for the `gh` CLI.
67 handle :: PullRequest -> GitHub.Handle
68 handle pr =
69   GitHub.Handle
70     { GitHub.timeline = readIORef (prTimeline pr),
71       GitHub.baseRef = pure (branch GitHub.Base),
72       GitHub.comments = unlines <$> readIORef (prComments pr),
73       GitHub.postComment = \body -> modifyIORef' (prComments pr) (++ [body])
74     }
75
76 -- | Push a version of the branch: the commit it forks off onto the branch the
77 -- pull request targets, its tip onto the branch under review.
78 --
79 -- Hand back the push to the branch under review, which is the one an action
80 -- fires on: 'Nothing' where that branch was already there, since a ref pushed
81 -- onto the sha it is already on moves nothing and no push happens at all.
82 push :: PullRequest -> CommitSha -> CommitSha -> IO (Maybe GitHub.Ev)
83 push pr base head' = do
84   _ <- move pr GitHub.Base base
85   move pr GitHub.Head head'
86
87 -- Push a sha onto one of the two branches, and say what push came of it. The
88 -- branch moves either way. The timeline only remembers the push where it was
89 -- not a fast-forward, since an ordinary push leaves no force-push event behind.
90 move :: PullRequest -> GitHub.Ref -> CommitSha -> IO (Maybe GitHub.Ev)
91 move pr ref sha = do
92   before <- readIORef (tip pr ref)
93   if before == sha
94     then pure Nothing
95     else do
96       let ev = GitHub.Ev ref before sha
97       fastForward <- Git.isAncestorOf (FakeGit.handle (prOrigin pr)) before sha
98       unless fastForward $ modifyIORef' (prTimeline pr) (++ [ev])
99       place pr ref sha
100      writeIORef (tip pr ref) sha

```

```

101 pure (Just ev)
102
103 -- | Its comments, oldest first.
104 comments :: PullRequest -> IO [String]
105 comments = readIORef . prComments

```

./test/GhPostRangeDiff/RenderSpec.hs [159 lines]

```

1 {-# OPTIONS_GHC -Wno-orphans #-}
2
3 module GhPostRangeDiff.RenderSpec (spec) where
4
5 import Data.List (isInfixOf, stripPrefix)
6 import Data.Maybe (mapMaybe)
7 import GhPostRangeDiff.Render (format)
8 import RangeDiff (Change (..), Commit (..), CommitMessage, Interdiff
9   (interdiffText), commitMessage, interdiff, messageText)
10 import RangeDiff.CommitSha (CommitSha, commitSha, shaText)
11 import Test.Hspec
12 import Test.QuickCheck
13
14 -- A commit message: printable words with the occasional run of backticks,
15 -- which sits mid-line and so must not be read as a code fence.
16 --
17 -- Nothing more is asked of it than that. A message goes out on the header line
18 -- and comes straight back off it, and 'commitMessage' has already dropped
19 -- everything past the first line, so whatever is left round-trips.
20 instance Arbitrary CommitMessage where
21   arbitrary = commitMessage . unwords <$> resize 3 (listOf word)
22   where
23     word = frequency [(4, getPrintableString <$> arbitrary), (1, pure "`")]
24
25 -- Shrink the text and read it back as a message. A candidate that comes back
26 -- as the message we started from would let shrinking loop, so those go.
27 shrink m = filter (/= m) (map commitMessage (shrink (messageText m)))
28
29 -- An interdiff: arbitrary text, built from chunks so that newlines and runs
30 -- of backticks both turn up often.
31 instance Arbitrary Interdiff where
32   arbitrary = interdiff . concat <$> listOf chunk
33   where
34     chunk =
35       oneof
36         [ getPrintableString <$> arbitrary,
37           pure "\n",
38           ('replicate` ``') <$> choose (1, 6)
39         ]
40
41 -- Shrinking the text can drop the trailing newline, which 'interdiff' then
42 -- puts straight back; dropping those candidates keeps shrinking from looping
43 -- on an unchanged value.
44 shrink x = filter (/= x) (map interdiff (shrink (interdiffText x)))
45
46 instance Arbitrary CommitSha where
47   arbitrary = vectorOf 7 (elements "0123456789abcdef") `suchThatMap` commitSha
48
49 -- A shrunk sha can be too short, or no longer hex; 'commitSha' drops those.
50 shrink = mapMaybe commitSha . shrink . shaText
51
52 instance Arbitrary Change where
53   arbitrary =
54     oneof
55       [ pure Added,
56         pure Removed,
57         pure Unchanged,
58         Updated <$> arbitrary
59       ]

```



```

59
60 -- A commit that carries no interdiff is the simpler counterexample, so try
61 -- that before shrinking the interdiff itself. The other three are as small
62 -- as a change gets.
63 shrink (Updated patch) = Unchanged : map Updated (shrink patch)
64 shrink _ = []
65
66 instance Arbitrary Commit where
67   arbitrary = Commit <$> arbitrary <*> arbitrary <*> arbitrary
68
69   shrink (Commit change sha message) =
70     [Commit change' sha message | change' <- shrink change]
71     ++ [Commit change sha' message | sha' <- shrink sha]
72     ++ [Commit change sha message' | message' <- shrink message]
73
74 data Block = PlainLineOfText String | CodeBlock String [String]
75
76 isBlank :: String -> Bool
77 isBlank = all (`elem` " \t")
78
79 -- | Check if a line counts as a fence. If it does, return the number of backticks and
80 -- the text after it. CommonMark allows up to three spaces of indent on these lines:
81 -- https://spec.commonmark.org/0.31.2/#fenced-code-blocks
82 matchFence :: Int -> String -> Maybe (Int, String)
83 matchFence minimalNumberOfBackticks l
84   | length indent <= 3, length ticks >= minimalNumberOfBackticks = Just
85     (length ticks, rest)
86   | otherwise = Nothing
87   where
88     (indent, l') = span (== ' ') l
89     (ticks, rest) = span (== '`') l'
90
91 -- | Split Markdown into blocks.
92 blocks :: [String] -> Either String [Block]
93 blocks [] = Right []
94 blocks (l : ls)
95   | isBlank l = blocks ls
96   | Just (n, language) <- matchFence 3 l,
97     '' `notElem` language =
98     case break (closes n) ls of
99       (body, _ : rest) -> (CodeBlock language body :) <$> blocks rest
100       (_, []) -> Left ("unterminated code fence: " ++ show l)
101   | otherwise = (PlainLineOfText l :) <$> blocks ls
102   where
103     -- A fence closes on the first line whose own run is at least as long,
104     -- followed by nothing but spaces or tabs.
105     closes n = maybe False (isBlank . snd) . matchFence n
106
107 data Tag = TagAdded | TagRemoved | TagUpdated | TagUnchanged
108
109 data Header = Header Tag CommitSha CommitMessage
110
111 parseTag :: String -> Either String (Tag, String)
112 parseTag l
113   | Just r <- tag "\128994 **Added**" = Right (TagAdded, r)
114   | Just r <- tag "\128308 **Removed**" = Right (TagRemoved, r)
115   | Just r <- tag "\128992 **Updated**" = Right (TagUpdated, r)
116   | Just r <- tag "\9898 **Unchanged**" = Right (TagUnchanged, r)
117   | otherwise = Left ("not a commit line: " ++ show l)
118   where
119     tag t = stripPrefix (t ++ " ") l
120
121 parseHeader :: String -> Either String Header
122 parseHeader l = do
123   (tag, rest) <- parseTag l
124   case break (== ' ') rest of

```

```

124 (s, l : message) -> case commitSha s of
125   Just sha -> Right (Header tag sha (commitMessage message))
126   Nothing -> Left ("commit line has no sha: " ++ show l)
127   _ -> Left ("commit line has no commit message: " ++ show l)
128
129 toCommits :: [Block] -> Either String [Commit]
130 toCommits [] = Right []
131 toCommits (PlainLineOfText l : bs) = do
132   Header tag sha message <- parseHeader l
133   let (change, bs') = case (tag, bs) of
134     -- A fenced diff belongs to the updated commit right above it.
135     (TagUpdated, CodeBlock "diff" body : rest) -> (Updated
136       (interdiff (unlines body)), rest)
137     (TagUpdated, _) -> (Updated (interdiff ""), bs)
138     (TagAdded, _) -> (Added, bs)
139     (TagRemoved, _) -> (Removed, bs)
140     (TagUnchanged, _) -> (Unchanged, bs)
141   (Commit change sha message :) <$> toCommits bs'
142 toCommits (CodeBlock info _ : _) = Left ("stray code block: " ++ show info)
143
144 -- | Read 'format' output back into the commits it was rendered from.
145 reparse :: String -> Either String [Commit]
146 reparse s = blocks (lines s) >=> toCommits
147
148 -- | Whether a three-backtick fence would have been closed by the interdiff
149 -- itself, so that `format` had to widen it.
150 widened :: Commit -> Bool
151 widened (Commit (Updated patch) _ _) = "```" `isInfixOf` interdiffText patch
152 widened _ = False
153
154 spec :: Spec
155 spec = describe "format" $
156   it "renders commits that can losslessly be converted back" $
157     checkCoverage $
158       forAllShrink (resize 8 (listOf arbitrary)) shrink $ \commits ->
159         cover 10 (any widened commits) "widened fence" $
160         reparse (format commits) === Right commits

```

./test/Spec.hs [1 lines]

```
1 {-# OPTIONS_GHC -F -pgmF hspec-discover #-}
```